

Manuel d'utilisation — `mir_esp_deploy`

Outil graphique de déploiement des capteurs ESP32 du projet MIR : flash du firmware + configuration des identifiants Wi-Fi (AP1 / AP2) via **liaison série (USB)** ou **BLE provisioning** (sans fil).

Fichiers concernés :

- `main.py` — point d'entrée Qt
- `esp32_deploy_view.py` — fenêtre (`Esp32DeployView`)
- `esp32_deploy_view_model.py` — logique (`Esp32DeployViewModel`)
- `ble_provisioning.py` — provisioning BLE (`BleProvisioner`, basé sur `bleak`)
- `bin/` — binaires flashés (`bootloader.bin`, `partitions.bin`, `boot_app0.bin`, `firmware.bin`)

1. À quoi ça sert ?

Besoin	Bouton	Canal
Mettre à jour le firmware de l'ESP32	Mettre à jour le firmware	Série USB uniquement
Relire les Wi-Fi mémorisés (<code>ap1_ssid</code> , <code>ap1_pwd</code> , <code>ap2_ssid</code> , <code>ap2_pwd</code>)	Lire config Wi-Fi	Série USB uniquement
Écrire les Wi-Fi primaire + secours	Appliquer config Wi-Fi	Série USB ou BLE
Changer de carte / port	Listes Port série + Modèle ESP32, Rafraîchir	—

Le firmware sait se connecter à deux points d'accès (AP1 prioritaire, AP2 secours). Seul AP1 est configurable en BLE ; AP1 + AP2 sont configurables en série.

2. Prérequis

2.1 Logiciels

- Python 3.10+ avec les packages MIR installés en mode éditable :

```
pip install -e ./mir_utils
pip install -e ./mir_devices
pip install -r requirements.txt # contient esptool
```

- Dépendances de `mir_esp_deploy` :
 - `PySide6` (interface)
 - `esptool` (flash) — `pip install esptool`
 - `bleak` (BLE, mode BLE uniquement) — `pip install bleak`
 - `pyserial` (via `mir_devices`)

- Bluetooth LE fonctionnel sur le PC pour le mode BLE (adaptateur + BlueZ sous Linux).

2.2 Matériel / OS

- ESP32 supportés : **esp32-wroom** (ex. ESP32-DevKit) et **esp32-cam** (voir §4.2).
- Câble USB **data** (pas un câble de charge seule).
- Driver USB-UART (CP210x / CH340) installé si besoin.
- Sous Linux, droits sur le port série :

```
sudo usermod -a -G dialout $USER
# puis se reconnecter. Dépannage : sudo chmod 666 /dev/ttyUSB0
```

- Sous Windows : repérer le **COMx** dans le Gestionnaire de périphériques.
- Sur Linux, **find_usb_serial_ports()** ne liste que **/dev/ttyUSB***. Sur Windows, tous les ports COM sont listés.

2.3 Binaires du firmware (**bin/**)

Le flash utilise obligatoirement les 4 fichiers de **mir_esp_deploy/bin/** :

```
0x1000  bootloader.bin
0x8000  partitions.bin
0xe000  boot_app0.bin
0x10000 firmware.bin
```

Ils sont régénérés automatiquement à chaque build PlatformIO du projet **esp32_sensor** par **esp32_sensor/sync_after_build.py** (post-action **buildprog**). Ne pas les éditer ni les déplacer. Si l'un manque, l'outil affiche **Bootloader / Partitions / Boot app0 / Firmware introuvable** et refuse de flasher.

Contrainte firmware : le BLE (Bluedroid) impose le partitionnement **no_ota.csv** (1 × 2 Mo APP, pas d'OTA). C'est déjà pris en compte dans **esp32_sensor/platformio.ini**.

3. Lancement

Depuis la racine du dépôt **mir/** :

```
python -m mir_esp_deploy.main
```

ou, pour debug, lancer **main.py** directement dans VS Code (workspace **mir.code-workspace** inclus).

Au démarrage la fenêtre « **Mir ESP Déploiement** » (580 × 620) s'ouvre, les ports série sont scannés automatiquement, la barre de statut affiche **N port(s) série trouvé(s)** ou **Aucun port série détecté**.

4. Description de l'interface

4.1 Groupe « Cible »

- **Radio Port série / BLE Provisioning** : sélectionne le mode (`DeployMode.SERIAL` / `DeployMode.BLE`).
 - En mode série : port + modèle actifs, flash / lecture / écriture série disponibles.
 - En mode BLE : port, modèle, flash et lecture désactivés ; seule **Appliquer config Wi-Fi** reste active (AP1 uniquement). Le label rappelle : « *Rebootez l'ESP32 avant d'appliquer la config (vous disposez de 60 s après le reboot)* ».
- **Liste Port série** : ports détectés. **Rafraîchir** relance `scan_ports()`. Si aucun port : (aucun port série détecté).
- **Liste Modèle ESP32** (= configuration série, libellé historique « baud ») :

Modèle	Baud flash	RTS/DTR	Vidage boot (<code>empty_loop</code>)
<code>esp32-wroom</code> (défaut)	921 600	oui	non
<code>esp32-cam</code>	1 000 000	non	oui (purge ~15 s de logs boot avant dialogue)

Choisir le modèle correspondant à la carte **avant** flash / lecture / écriture. La lecture/écriture série dialogue ensuite à 115 200 bauds.

4.2 Groupe « Identifiants Wi-Fi »

- **AP1 SSID / AP1 PWD** : réseau prioritaire.
- **AP2 SSID / AP2 PWD** : réseau secours (série uniquement).
- Champs mot de passe avec bouton œil (afficher / masquer).
- Bouton **↔ (swap)** : échange AP1 ↔ AP2.
- En mode BLE, AP2 est grisé : seuls AP1 SSID + AP1 PWD sont envoyés. SSID vide ou mot de passe vide = configuration rejetée en BLE (**ValueError: Configuration Wi-Fi invalide**). Côté firmware, SSID tronqué à 32 car., PWD à 64 car. ; PWD vide = réseau ouvert (mais la caractéristique PWD doit quand même être écrite au moins une fois — l'outil l'écrit toujours).

4.3 Groupe « Actions »

Bouton	Condition d'activation	Effet
Mettre à jour le firmware	Mode série + port choisi + <code>bin/* .bin</code> présents + pas d'opération en cours	Efface le log, lance <code>esptool write-flash ...</code> en tâche de fond
Lire config Wi-Fi	Mode série + port + libre	<code>get_ap_configuration</code> via série, remplit les 4 champs
Appliquer config Wi-Fi	Libre (série : + port)	Série : <code>set_ap_configuration ...</code> ; BLE : <code>BleProvisioner.set_ap(ssid, pwd)</code>

Pendant toute opération, les contrôles sont grisés (**busy**), la fermeture de la fenêtre est bloquée, la barre de statut indique l'étape en cours.

4.4 Groupe « Logs » + barre de statut

- **QPlainTextEdit** monospace, lecture seule : sortie **esptool** ligne par ligne (codes couleur ANSI nettoyés) + messages **Connexion port série...**, **Lecture / Ecriture configuration ap...**, erreurs.
- Le bouton flash vide le log avant de commencer.
- Barre de statut en bas : **Prêt**, **Flash en cours...**, **Le firmware a été flashé avec succès**, **Configuration Wi-Fi lue / envoyée...**, erreurs, etc.
- Les exceptions non gérées sont capturées (**sys.excepthook**) et affichées dans une boîte de dialogue + **stderr**.

5. Procédures pas à pas

5.1 Flasher le firmware (série, obligatoire pour la 1^{re} installation)

1. Sélectionner **Port série**.
2. Brancher l'ESP32 en USB, cliquer **Rafraîchir**, choisir le port (**/dev/ttyUSB0** sous Linux, **COMx** sous Windows).
3. Choisir le **Modèle ESP32** (**esp32-wroom** sauf ESP32-CAM).
4. Cliquer **Mettre à jour le firmware**.
5. Suivre la progression dans **Logs**. Commande réellement exécutée :

```
esptool --chip esp32 --port <port> --baud <921600|1000000> \
--before default-reset --after hard-reset write-flash -z \
--flash-mode dio --flash-freq 40m --flash-size 4MB \
0x1000 bootloader.bin 0x8000 partitions.bin \
0xe000 boot_app0.bin 0x10000 firmware.bin
```

6. Attendre le dialogue « **Flash terminé** ». Code retour **0** = succès ; sinon le message précise le code retour.
7. L'ESP32 redémarre seul (**hard-reset**).

En cas d'échec : vérifier câble, port, droits, qu'aucun autre logiciel (moniteur série, autre instance) n'utilise le port, et que les 4 **.bin** existent. Baisser le débit n'est pas exposé dans l'UI (modifier **SerialConfig** dans **esp32_deploy_view_model.py** si besoin).

5.2 Lire la configuration Wi-Fi (série)

1. Mode **Port série**, port + modèle corrects.
2. Cliquer **Lire config Wi-Fi**.
3. Les 4 champs sont remplis automatiquement + dialogue de confirmation. Si rien n'est reçu : avertissement **L'ESP32 n'a pas renvoyé de configuration valide** (vérifier que le firmware MIR tourne et que le modèle série — surtout **empty_loop** pour ESP32-CAM — est correct).

Protocole sous-jacent : `get_ap_configuration\n` → `#get_ap_configuration`
`ap1_ssid=...;ap1_pwd=...;...`

5.3 Écrire la configuration Wi-Fi en série (AP1 + AP2)

1. Mode **Port série**, port + modèle corrects.
2. Renseigner AP1 (obligatoire) et AP2 (optionnel mais recommandé comme secours).
3. Cliquer **Appliquer config Wi-Fi**.
4. Dialogue « **Les identifiants Wi-Fi ont été écrits sur l'ESP32** » en cas de `status=success`, sinon erreur `L'ESP32 n'a pas confirmé l'écriture`.

Protocole : `set_ap_configuration ap1_ssid=...;ap1_pwd=...;ap2_ssid=...;ap2_pwd=...` (écriture NVS côté ESP32, prise en compte au reboot / reconnexion). Note : par sécurité, `set_ap_configuration` est refusé en TCP ; seul le série (et le BLE pour AP1) est autorisé.

5.4 Écrire la configuration Wi-Fi en BLE (sans fil, AP1 seul)

Préconditions : firmware MIR déjà flashé avec BLE, PC avec Bluetooth LE, `pip install bleak`, ESP32 **redémarré depuis moins de 60 s**.

1. Sélectionner **BLE Provisioning**.
2. **Rebooter l'ESP32** (bouton RST / cycle d'alimentation).
3. Renseigner **AP1 SSID + AP1 PWD** (non vides). AP2 est ignoré.
4. Dans les 60 s après le reboot, cliquer **Appliquer config Wi-Fi**.
5. Suivre les logs : `Connexion BLE en cours...` puis `Ecriture configuration ap...`.
6. Succès = dialogue de confirmation. L'ESP32 envoie `SUCCESS:rebooting`, coupe le BLE et **reboote** sur le nouveau réseau — la déconnexion BLE qui suit est normale.

Détails protocole (cf. `doc/esp32/esp_sensors_protocol.md` §5) :

- Nom advertised : `MIR_ESP_SENSOR`, service `e0f0c9a0-...-01`.
- `WRITE SSID (...02)`, `WRITE PWD (...03)`, `READ|NOTIFY STATUS (...04)`.
- STATUS : `WAIT_CREDENTIALS` → `RECEIVED_SSID / RECEIVED_PWD` → `SUCCESS:rebooting`.
- Se connecter pendant la fenêtre de 60 s **fige** le BLE en mode persistant (plus de timeout). Sans connexion ni écriture pendant 60 s, le BLE s'arrête (`BLE provisioning : aucune demande reçue pendant 60 s, arrêt`) et il faut rebooter pour réessayer.
- Erreur `équipement non trouvé (hors fenêtre 60 s après boot ?)` = redémarrer l'ESP32 et recommencer dans les 60 s ; vérifier proximité, Bluetooth activé, pas déjà connecté ailleurs.
- Aucun chiffrement BLE : SSID/PWD transitent en clair, comme en série. Réservé au provisioning de proximité.

Astuce debug : le même provisioning est testable en CLI ou avec nRF Connect (activer les notifications sur STATUS `...04`, écrire SSID puis PWD) :

```
python mir_esp_deploy/ble_provisioning.py
# connecté : <adresse>
# status initial : WAIT_CREDENTIALS
# provisioning : OK (reboot en cours)
```

6. Dépannage

Symptôme	Cause probable / action
(aucun port série détecté)	Câble charge seule, driver manquant, droits Linux (dialout), carte non branchée → Rafraîchir après correction
Port non sélectionné	Cliquer Rafraîchir et choisir un port
Bootloader / Partitions / Boot app0 / Firmware introuvable	Rebuild esp32_sensor sous PlatformIO (la synchro vers bin/ est automatique)
esptool introuvable / Failed to connect	pip install esptool , bon port, bon modèle (esp32-cam vs wroom), fermer moniteurs série concurrents, maintenir BOOT si carte sans auto-reset
L'ESP32 n'a pas renvoyé de configuration valide / timeout série	Mauvais modèle (notamment empty_loop pour CAM), firmware non MIR, ESP32 en boot-loop, câblage
L'ESP32 n'a pas confirmé l'écriture	Refaire Lire puis Appliquer ; vérifier caractères spéciaux ; / = dans SSID/PWD (format k=v ; ...)
équipement non trouvé (hors fenêtre 60 s...) en BLE	Rebooter l'ESP32, cliquer Appliquer dans les 60 s, rapprocher, vérifier bleak + Bluetooth
Configuration Wi-Fi invalide en BLE	Renseigner AP1 SSID et AP1 PWD non vides
Fenêtre qui ne se ferme pas pendant une opération	Normal : fermeture bloquée tant que busy ; attendre la fin
AP2 non pris en compte en BLE	Normal : BLE = AP1 seul. Utiliser le mode série pour AP2

Logs utiles : panneau **Logs** de l'outil + **logs/** à la racine du dépôt (**enable_logging()** de **mir_utils**).

7. Sécurité et limites

- get_ap_configuration / set_ap_configuration** refusés en TCP (**serial_only_command**) : configuration uniquement en série ou BLE.
- BLE sans appairage/chiffrement : mots de passe en clair sur la radio locale — usage de proximité uniquement, comme la liaison série.
- Les logs série firmware n'affichent que les **longueurs** SSID/PWD, jamais les valeurs en clair.
- Mot de passe vide accepté côté firmware (réseau ouvert) mais l'outil exige un PWD non vide en mode BLE ; utiliser le mode série pour un réseau ouvert.
- Pas d'OTA : toute mise à jour de firmware passe par USB + **esptool**.

8. Pour développeurs : rebuild et extension

- Rebuild firmware : ouvrir `esp32_sensor/` sous PlatformIO → Build. `sync_after_build.py` copie `bootloader.bin`, `partitions.bin`, `firmware.bin` (+ `boot_app0.bin` du framework Arduino) vers `mir_esp_deploy/bin/`.
- Ajouter un modèle de carte : ajouter une entrée à `_serial_configurations` dans `esp32_deploy_view_model.py` (`flash_baud_rate`, `enable_rts_dtr`, `empty_loop`).
- Réutiliser le BLE en script :

```
from mir_esp_deploy.ble_provisioning import BleProvisioner
with BleProvisioner() as prov:
    prov.connect(timeout=20.0)
    ok = prov.set_ap("MonAP", "MonPwd")
```